

Week-6-Assignment

ParkingLot – Requirement-1

Code:

Program.cs

```
// ParkingLot - Requirement 1

namespace Requirement_1
{
    internal class Program
    {
        // Reads a Single line of input
        static Vehicle ReadVehicle()
        {
            try
            {
                // 7) The input format consists of Vehicle details separated by
                comma
                string[] data = Console.ReadLine().Split(',');
                string registrationNo = data[0];
                string name = data[1];
                string type = data[2];
                double.TryParse(data[3], out double weight);
                string ticketNo = data[4];
                DateTime.TryParse(data[5], out DateTime parkedTime);
                double.TryParse(data[6], out double cost);

                Ticket ticket = new Ticket(ticketNo, parkedTime, cost);
                return new Vehicle(registrationNo, name, type, weight, ticket);
            }
            // Handles parsing and format related issues
            catch
            {
                Console.WriteLine("Invalid input format. Please enter correct
                vehicle details.");
                throw;
            }
        }
        static void Main(string[] args)
        {
            try
            {
                // Enter details for first vehicle
                Console.WriteLine("Enter Vehicle 1 details:");
                Vehicle v1 = ReadVehicle();

                // Enter details for second vehicle
                Console.WriteLine("Enter Vehicle 2 details:");
                Vehicle v2 = ReadVehicle();

                Console.WriteLine();

                // Display details of Vehicle 1
                Console.WriteLine("Vehicle 1");
                Console.WriteLine(v1);
                Console.WriteLine();

                // Display details of Vehicle 2
                Console.WriteLine("Vehicle 2");
            }
        }
    }
}
```

```

        Console.WriteLine(v2);
        Console.WriteLine();

        // 8) Compare both vehicle objects
        if (v1.Equals(v2))
            Console.WriteLine("Vehicle 1 is same as Vehicle 2");
        else
            Console.WriteLine("Vehicle 1 is different from Vehicle 2");
    }
    // Handles parsing and format related issues
    catch (FormatException)
    {
        Console.WriteLine("Error occurred while reading vehicle
details.");
    }
    catch (Exception)
    {
        Console.WriteLine("An unexpected error occurred.");
    }
}
}
}

```

Vehicle.cs

```

namespace Requirement_1
{
    // 1) Create a Vehicle Class
    internal class Vehicle
    {
        // Private fields to store Vehicle details
        private string _registrationNo;
        private string _name;
        private string _type;
        private double _weight;
        private Ticket _ticket;

        // 3) Create / Generate appropriate Properties

        // Public property to get/set registration number
        public string RegistrationNo
        {
            get { return _registrationNo; }
            set { _registrationNo = value; }
        }

        // Public property to get/set vehicle name
        public string Name
        {
            get { return _name; }
            set { _name = value; }
        }

        // Public property to get/set vehicle type
        public string Type
        {
            get { return _type; }
            set { _type = value; }
        }

        // Public property to get/set vehicle weight
        public double Weight

```

```

    {
        get { return _weight; }
        set { _weight = value; }
    }

    // Public property to get/set ticket details
    public Ticket Ticket
    {
        get { return _ticket; }
        set { _ticket = value; }
    }

    // 4) Parameterized constructor to initialize all vehicle attributes
    public Vehicle(string? registrationNo, string? name, string? type, double
weight, Ticket ticket)
    {
        _registrationNo = registrationNo;
        _name = name;
        _type = type;
        _weight = weight;
        _ticket = ticket;
    }

    // 5) Override ToString() to display vehicle details in required format
    public override string ToString()
    {
        return $"Registration No: {_registrationNo}\n" +
            $"Name: {_name}\n" +
            $"Type: {_type}\n" +
            $"Weight: {_weight}\n" +
            $"Ticket Details: {_ticket}";
    }

    // 6) Override Equals() to compare two Vehicle objects
    public override bool Equals(object obj)
    {
        if (obj == null || obj is not Vehicle)
            return false;

        Vehicle other = (Vehicle)obj;

        return string.Equals(this._registrationNo,
other._registrationNo, StringComparison.OrdinalIgnoreCase)
            && string.Equals(this._name,
other._name, StringComparison.OrdinalIgnoreCase);
    }

    // Override GetHashCode()
    public override int GetHashCode()
    {
        return (_registrationNo.ToLower() + _name.ToLower()).GetHashCode();
    }
}

```

Ticket.cs

```
namespace Requirement_1
{
    // 2) Create a Ticket Class
    internal class Ticket
    {
        // Private field to store ticket details
        private string _ticketNo;
        private DateTime _parkedTime;
        private double _cost;

        // 3) Create / Generate appropriate Properties

        // Public property to get/set ticket number
        public string TicketNo
        {
            get { return _ticketNo; }
            set { _ticketNo = value; }
        }

        // Public property to get/set parked time
        public DateTime ParkedTime
        {
            get { return _parkedTime; }
            set { _parkedTime = value; }
        }

        // Public property to get/set parking cost
        public double Cost
        {
            get { return _cost; }
            set { _cost = value; }
        }

        // 4) Parameterized constructor to initialize ticket details
        public Ticket(string ticketNo, DateTime parkedTime, double cost)
        {
            _ticketNo = ticketNo;
            _parkedTime = parkedTime;
            _cost = cost;
        }

        // 5) Override ToString() to display ticket number in required format
        public override string ToString()
        {
            return _ticketNo;
        }
    }
}
```

Outputs:

Different Vehicle

```
Enter Vehicle 1 details:
AP 19 QS 4556,R3,TwoWheeler,196,A1-002,10-05-2018 11:05:21,100
Enter Vehicle 2 details:
MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75

Vehicle 1
Registration No: AP 19 QS 4556
Name: R3
Type: TwoWheeler
Weight: 196
Ticket Details: A1-002

Vehicle 2
Registration No: MH 23 F 7856
Name: Gixer
Type: TwoWheeler
Weight: 221
Ticket Details: A1-013

Vehicle 1 is different from Vehicle 2
```

Same Vehicle

```
Enter Vehicle 1 details:
KL 75 F 9562,Pulsar,TwoWheeler,200,A1-032,12-05-2018 14:25:23,50
Enter Vehicle 2 details:
KL 75 F 9562,Pulsar,TwoWheeler,200,A1-032,12-05-2018 14:25:23,50

Vehicle 1
Registration No: KL 75 F 9562
Name: Pulsar
Type: TwoWheeler
Weight: 200
Ticket Details: A1-032

Vehicle 2
Registration No: KL 75 F 9562
Name: Pulsar
Type: TwoWheeler
Weight: 200
Ticket Details: A1-032

Vehicle 1 is same as Vehicle 2
```

ParkingLot – Requirement-2

Code:

Program.cs

```
// ParkingLot - Requirement 2

namespace Requirement_2
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.Write("Enter the name of the Parking Lot: ");
                string parkingLotName = Console.ReadLine();

                // Create ParkingLot object
                ParkingLot parkingLot = new ParkingLot(parkingLotName, new
List<Vehicle>());

                int choice;
                do
                {
                    // Display menu
                    Console.WriteLine("\n1.Add Vehicle");
                    Console.WriteLine("2.Delete Vehicle");
                    Console.WriteLine("3.Display Vehicles");
                    Console.WriteLine("4.Exit");
                    Console.WriteLine("Enter -1 to Exit\n");
                    Console.Write("Enter your choice: ");

                    int.TryParse(Console.ReadLine(), out choice);

                    switch (choice)
                    {
                        // Add Vehicle
                        case 1:
                            Console.Write("\nEnter Vehicle details: ");
                            string input = Console.ReadLine();

                            // Create vehicle using static method
                            Vehicle vehicle = Vehicle.CreateVehicle(input);

                            parkingLot.AddVehicleToParkingLot(vehicle);
                            Console.WriteLine("Vehicle successfully added");
                            break;

                        // Delete Vehicle
                        case 2:
                            Console.Write("Enter Regsitration Number: ");
                            string regNo = Console.ReadLine();

                            bool result =
parkingLot.RemoveVehicleFromParkingLot(regNo);

                            if (result)
                                Console.WriteLine("Vehicle successfully
deleted");
                            else

```

```

        Console.WriteLine("Vehicle not found in
parkinglot");
        break;

        // Display Vehicles
        case 3: parkingLot.DisplayVehicles(); break;

        // Exit
        case -1: Console.WriteLine("Exiting Program..."); return;

        default: Console.WriteLine("Invalid choice"); break;
    }
    } while (choice != -1);
}
// To print Exception message
catch(Exception e)
{
    Console.WriteLine(e.Message);
}
}
}
}

```

Vehicle.cs

```

namespace Requirement_2
{
    // a) Create a Class Vehicle
    internal class Vehicle
    {
        // Private fields to store Vehicle details
        private string _registrationNo;
        private string _name;
        private string _type;
        private double _weight;
        private Ticket _ticket;

        // Public property to get/set registration number
        public string RegistrationNo
        {
            get { return _registrationNo; }
            set { _registrationNo = value; }
        }

        // Public property to get/set vehicle name
        public string Name
        {
            get { return _name; }
            set { _name = value; }
        }

        // Public property to get/set vehicle type
        public string Type
        {
            get { return _type; }
            set { _type = value; }
        }

        // Public property to get/set vehicle weight
        public double Weight
        {
            get { return _weight; }

```

```

        set { _weight = value; }
    }

    // Public property to get/set ticket details
    public Ticket Ticket
    {
        get { return _ticket; }
        set { _ticket = value; }
    }

    // Parameterized constructor to initialize all vehicle attributes
    public Vehicle(string? registrationNo, string? name, string? type, double
weight, Ticket ticket)
    {
        _registrationNo = registrationNo;
        _name = name;
        _type = type;
        _weight = weight;
        _ticket = ticket;
    }

    // d) Create Vehicle
    public static Vehicle CreateVehicle(string detail)
    {
        // Split the input string
        string[] data = detail.Split(',');

        // Vehicle details
        string registrationNo = data[0];
        string name = data[1];
        string type = data[2];
        double weight = double.Parse(data[3]);

        // Ticket details
        string ticketNo = data[4];
        DateTime parkedTime = DateTime.Parse(data[5]);
        double cost = double.Parse(data[6]);

        // Create Ticket object
        Ticket ticket = new Ticket(ticketNo, parkedTime, cost);

        // Create and return Vehicle object
        return new Vehicle(registrationNo, name, type, weight, ticket);
    }
}

```

Ticket.cs

```

namespace Requirement_2
{
    // b) Create a Class Ticket
    internal class Ticket
    {
        // Private field to store ticket details
        private string _ticketNo;
        private DateTime _parkedTime;
        private double _cost;

        // Public property to get/set ticket number
        public string TicketNo

```



```

    {
        get { return _ticketNo; }
        set { _ticketNo = value; }
    }

    // Public property to get/set parked time
    public DateTime ParkedTime
    {
        get { return _parkedTime; }
        set { _parkedTime = value; }
    }

    // Public property to get/set parking cost
    public double Cost
    {
        get { return _cost; }
        set { _cost = value; }
    }

    // Parameterized constructor to initialize ticket details
    public Ticket(string ticketNo, DateTime parkedTime, double cost)
    {
        _ticketNo = ticketNo;
        _parkedTime = parkedTime;
        _cost = cost;
    }
}

```

ParkingLot.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Requirement_2
{
    // c) Create a Class ParkingLot
    internal class ParkingLot
    {
        // Private fields to store ParkingLot details
        private string _name;
        private List<Vehicle> _vehicleList;

        //Public property to get/set parking lot name
        public String Name
        {
            get { return _name; }
            set { _name = value; }
        }

        //Public property to get/set Vehicle List
        public List<Vehicle> VehicleList
        {
            get { return _vehicleList; }
            set { _vehicleList = value; }
        }

        // Default Constructor
    }
}

```

```

public ParkingLot()
{
    _vehicleList = new List<Vehicle>();
}

// Parameterized Constructor
public ParkingLot(string _name, List<Vehicle> _vehicleList)
{
    this._name = _name;
    this._vehicleList = new List<Vehicle>();
}

// e) Create the following methods in ParkingLot class
// Add Vehicle
public void AddVehicleToParkingLot(Vehicle vehicle)
{
    _vehicleList.Add(vehicle);
}

// Remove Vehicle
public bool RemoveVehicleFromParkingLot(string registrationNo)
{
    foreach (Vehicle v in _vehicleList)
    {
        if (string.Equals(v.RegistrationNo, registrationNo,
StringComparison.OrdinalIgnoreCase))
        {
            _vehicleList.Remove(v);
            return true;
        }
    }
    return false;
}

// Display Vehicles in parking lot
public void DisplayVehicles()
{
    if (_vehicleList.Count == 0)
    {
        Console.WriteLine("No vehicles to show");
        return;
    }

    Console.WriteLine($"\\nVehicles in {Name}\\n");
    Console.Write("{0,-20} {1,-10} {2,-12} {3,-7} {4}\\n",
        "Registration No", "Name", "Type", "Weight", "Ticket
No");

    foreach (Vehicle v in _vehicleList)
    {
        Console.Write("{0,-20} {1,-10} {2,-12} {3,-7:F1} {4}\\n",
            v.RegistrationNo,
            v.Name,
            v.Type,
            v.Weight,
            v.Ticket.TicketNo);
    }
}
}
}

```

Outputs:

Add Vehicle

```
Enter the name of the Parking Lot: A1

1.Add Vehicle
2.Delete Vehicle
3.Display Vehicles
4.Exit
Enter -1 to Exit

Enter your choice: 1

Enter Vehicle details: TN 56 AS 5489,R15,TwoWheeler,150,A1-001,11-05-2018 15:05:23,50
Vehicle successfully added

1.Add Vehicle
2.Delete Vehicle
3.Display Vehicles
4.Exit
Enter -1 to Exit

Enter your choice: 1

Enter Vehicle details: KL 75 F 9562,Pulsar,TwoWheeler,200,A1-032,12-05-2018 14:25:23,50
Vehicle successfully added
```

Delete Vehicle

```
1.Add Vehicle
2.Delete Vehicle
3.Display Vehicles
4.Exit
Enter -1 to Exit

Enter your choice: 2
Enter Regsitration Number: TN 56 AS 5489
Vehicle successfully deleted

1.Add Vehicle
2.Delete Vehicle
3.Display Vehicles
4.Exit
Enter -1 to Exit

Enter your choice: 3

Vehicles in A1

Registration No      Name      Type      Weight      Ticket No
KL 75 F 9562        Pulsar    TwoWheeler 200.0       A1-032
```

Display Vehicles

```
1.Add Vehicle
2.Delete Vehicle
3.Display Vehicles
4.Exit
Enter -1 to Exit
```

Enter your choice: 3

Vehicles in A1

Registration No	Name	Type	Weight	Ticket No
TN 56 AS 5489	R15	TwoWheeler	150.0	A1-001
KL 75 F 9562	Pulsar	TwoWheeler	200.0	A1-032

Vehicle Not Found to Delete

```
1.Add Vehicle
2.Delete Vehicle
3.Display Vehicles
4.Exit
Enter -1 to Exit
```

Enter your choice: 2

Enter Regsitration Number: TS 11 EA 7894

Vehicle not found in parkinglot

ParkingLot – Requirement-3

Code:

Program.cs

```
// Parking Lot – Requirement-3

using System.Text.RegularExpressions;
namespace Requirement_3
{
    internal class Program
    {
        // a) ValidateRegistrationNo Method
        static bool ValidateRegistrationNo(string registrationNo)
        {
            // b) Regex (Rules)
            string pattern = @"^[A-Z]{2}\s\d{1,2}(\s[A-Z]{1,2})?\s\d{1,4}$";

            return Regex.IsMatch(registrationNo, pattern);
        }
        static void Main(string[] args)
        {
            try
            {
                // Enter Registration number
                Console.WriteLine("Enter the registration no. to be validated: ");
                string registrationNo = Console.ReadLine();

                bool isValid = ValidateRegistrationNo(registrationNo);

                if (isValid)
                {
                    Console.WriteLine("Registration No. is valid");
                }
                else
                {
                    Console.WriteLine("Registration No. is invalid");
                }
                // Handles empty or invalid input safely
                catch (ArgumentException ex)
                {
                    Console.WriteLine(ex.Message);
                }
                catch (Exception)
                {
                    Console.WriteLine("An unexpected error occurred");
                }
            }
        }
    }
}
```

Outputs:

Valid Registration Number

```
Enter the registration no. to be validated: TS 01 K 1562  
Registration No. is valid
```

```
Enter the registration no. to be validated: TS 02 AB 9651  
Registration No. is valid
```

Invalid Registration Number

```
Enter the registration no. to be validated: UP 458 H 5462  
Registration No. is invalid
```

```
Enter the registration no. to be validated: Ts 12 EA 987  
Registration No. is invalid
```

ParkingLot – Requirement-4

Code:

Program.cs

// Parking Lot - Requirement-4

```
namespace Requirement_4
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                VehicleBO vehicleBO = new VehicleBO();
                List<Vehicle> vehicleList = new List<Vehicle>();

                // Read number of vehicles
                Console.WriteLine("Enter the number of vehicles: ");
                int.TryParse(Console.ReadLine(), out int n);

                // Read vehicle details
                Console.WriteLine("\nEnter Vehicle Details: ");
                for (int i = 0; i < n; i++)
                {
                    string input = Console.ReadLine();
                    Vehicle v = Vehicle.CreateVehicle(input);
                    vehicleList.Add(v);
                }

                // Search menu
                Console.WriteLine("\nEnter a search type:");
                Console.WriteLine("1.By type");
                Console.WriteLine("2.By parked time");

                int.TryParse(Console.ReadLine(), out int choice);

                List<Vehicle> result = new List<Vehicle>();

                //Search by Vehicle Type
                if (choice == 1)
                {
                    Console.WriteLine("\nEnter the vehicle type: ");
                    string type = Console.ReadLine();

                    result = vehicleBO.FindVehicle(vehicleList, type);
                }
                // Search by Parked time
                else if (choice == 2)
                {
                    Console.WriteLine("\nEnter the parked time (dd-MM-yyyy HH:mm:ss):");
                    DateTime parkedTime = DateTime.ParseExact(Console.ReadLine(),
                                                                "dd-MM-yyyy HH:mm:ss",
                                                                null);

                    result = vehicleBO.FindVehicle(vehicleList, parkedTime);
                }
                // Display result
                if (result.Count == 0)
                {

```

```

        Console.WriteLine("No vehicles found");
    }
    else
    {
        Console.WriteLine("\n{0,-20} {1,-10} {2,-12} {3,-7} {4}",
            "Registration No", "Name", "Type", "Weight", "Ticket No"
        );

        foreach (Vehicle v in result)
        {
            Console.WriteLine(
                "{0,-20} {1,-10} {2,-12} {3,-7:F1} {4}",
                v.RegistrationNo,
                v.Name,
                v.Type,
                v.Weight,
                v.Ticket.TicketNo
            );
        }
    }

    // Invalid inputs
    catch (FormatException)
    {
        Console.WriteLine("Invalid input format. Please check numeric and
date values.");
    }

    // Null or empty arguments
    catch (ArgumentException ex)
    {
        Console.WriteLine(ex.Message);
    }

    // Some other Exceptions
    catch (Exception)
    {
        Console.WriteLine("An unexpected error occurred.");
    }
}
}
}

```

Vehicle.cs

```

using System.Net.Sockets;
using System.Security.Claims;

namespace Requirement_4
{
    // a) Create a Class Vehicle
    internal class Vehicle
    {
        // Private fields to store Vehicle details
        private string _registrationNo;
        private string _name;
        private string _type;
        private double _weight;
        private Ticket _ticket;

        // Public property to get/set registration number
        public string RegistrationNo
        {
            get { return _registrationNo; }
        }
    }
}

```



```

        set { _registrationNo = value; }
    }

    // Public property to get/set vehicle name
    public string Name
    {
        get { return _name; }
        set { _name = value; }
    }

    // Public property to get/set vehicle type
    public string Type
    {
        get { return _type; }
        set { _type = value; }
    }

    // Public property to get/set vehicle weight
    public double Weight
    {
        get { return _weight; }
        set { _weight = value; }
    }

    // Public property to get/set ticket details
    public Ticket Ticket
    {
        get { return _ticket; }
        set { _ticket = value; }
    }

    // Default constructor
    public Vehicle()
    {
        _registrationNo = string.Empty;
        _name = string.Empty;
        _type = string.Empty;
        _weight = 0.0;
        _ticket = null;
    }

    // Parameterized constructor
    public Vehicle(string? registrationNo, string? name, string? type, double
weight, Ticket ticket)
    {
        _registrationNo = registrationNo;
        _name = name;
        _type = type;
        _weight = weight;
        _ticket = ticket;
    }

    // Create Vehicle
    public static Vehicle CreateVehicle(string detail)
    {
        // Split the input string
        string[] data = detail.Split(',');

        // Vehicle details
        string registrationNo = data[0];
        string name = data[1];
        string type = data[2];
        double weight = double.Parse(data[3]);
    }

```

```

        // Ticket details
        string ticketNo = data[4];
        DateTime parkedTime = DateTime.Parse(data[5]);
        double cost = double.Parse(data[6]);

        // Create Ticket object
        Ticket ticket = new Ticket(ticketNo, parkedTime, cost);

        // Create and return Vehicle object
        return new Vehicle(registrationNo, name, type, weight, ticket);
    }
}

```

Ticket.cs

```

using System.Security.Claims;

namespace Requirement_4
{
    // b) Create a Class Ticket
    internal class Ticket
    {
        // Private field to store ticket details
        private string _ticketNo;
        private DateTime _parkedTime;
        private double _cost;

        // Public property to get/set ticket number
        public string TicketNo
        {
            get { return _ticketNo; }
            set { _ticketNo = value; }
        }

        // Public property to get/set parked time
        public DateTime ParkedTime
        {
            get { return _parkedTime; }
            set { _parkedTime = value; }
        }

        // Public property to get/set parking cost
        public double Cost
        {
            get { return _cost; }
            set { _cost = value; }
        }

        // Default constructor
        public Ticket()
        {
            _ticketNo = string.Empty;
            _parkedTime = DateTime.MinValue;
            _cost = 0.0;
        }

        // Parameterized constructor
        public Ticket(string ticketNo, DateTime parkedTime, double cost)
        {

```

```

        _ticketNo = ticketNo;
        _parkedTime = parkedTime;
        _cost = cost;
    }
}

```

VehicleBO.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Requirement_4
{
    // c) Create a class VehicleBO
    internal class VehicleBO
    {
        // Find Vehicle by type
        public List<Vehicle> FindVehicle(List<Vehicle> vehicleList, string type)
        {
            List<Vehicle> result = new List<Vehicle>();

            foreach (Vehicle v in vehicleList)
            {
                // Comparison of vehicle Type
                if (string.Equals(v.Type, type,
StringComparison.OrdinalIgnoreCase))
                {
                    result.Add(v);
                }
            }
            return result;
        }
        // Find Vehicle by Parked time
        public List<Vehicle> FindVehicle(List<Vehicle> vehicleList, DateTime
parkedTime)
        {
            List<Vehicle> result = new List<Vehicle>();

            foreach (Vehicle v in vehicleList)
            {
                // Compare parked time
                if (v.Ticket.ParkedTime == parkedTime)
                {
                    result.Add(v);
                }
            }
            return result;
        }
    }
}

```

Outputs:

Search By Date Time

Enter the number of vehicles: 6

Enter Vehicle Details:

AP 19 QS 4556,R3,TwoWheeler,196,A1-002,10-05-2018 11:05:21,100
MP 01 LK 0001,Hornet,TwoWheeler,163,A1-009,09-05-2018 08:13:24,50
MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75
GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,11-05-2018 05:21:40,75
GJ 83 AX 0545,Ciaz,FourWheeler,530,B4-030,10-05-2018 08:25:33,150
HR 46 S 4523,Swift,FourWheeler,846,B4-021,09-05-2018 07:14:13,200

Enter a search type:

- 1.By type
 - 2.By parked time
- 2

Enter the parked time (dd-MM-yyyy HH:mm:ss): 11-05-2018 05:21:40

Registration No	Name	Type	Weight	Ticket No
MH 23 F 7856	Gixer	TwoWheeler	221.0	A1-013
GA 45 RF 9515	Duke	TwoWheeler	240.0	A1-024

Search By Vehicle Type

Enter the number of vehicles: 6

Enter Vehicle Details:

AP 19 QS 4556,R3,TwoWheeler,196,A1-002,10-05-2018 11:05:21,100
MP 01 LK 0001,Hornet,TwoWheeler,163,A1-009,09-05-2018 08:13:24,50
MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75
GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,10-05-2018 05:16:26,75
GJ 83 AX 0545,Ciaz,FourWheeler,530,B4-030,10-05-2018 08:25:33,150
HR 46 S 4523,Swift,FourWheeler,846,B4-021,09-05-2018 07:14:13,200

Enter a search type:

- 1.By type
 - 2.By parked time
- 1

Enter the vehicle type: FourWheeler

Registration No	Name	Type	Weight	Ticket No
GJ 83 AX 0545	Ciaz	FourWheeler	530.0	B4-030
HR 46 S 4523	Swift	FourWheeler	846.0	B4-021

ParkingLot – Requirement-5

Code:

Program.cs

// Parking Lot - Requirement-5

```
namespace Requirement_5
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                List<Vehicle> vehicleList = new List<Vehicle>();

                // Read number of vehicles
                Console.WriteLine("Enter the number of vehicles: ");
                int.TryParse(Console.ReadLine(), out int n);

                // Read vehicle details
                Console.WriteLine("\nEnter Vehicle Details: ");
                for (int i = 0; i < n; i++)
                {
                    string input = Console.ReadLine();
                    Vehicle v = Vehicle.CreateVehicle(input);
                    vehicleList.Add(v);
                }

                // Search menu
                Console.WriteLine("\nEnter a type to sort:");
                Console.WriteLine("1.Sort by weight");
                Console.WriteLine("2.Sort by parked time");

                int.TryParse(Console.ReadLine(), out int choice);

                List<Vehicle> result = new List<Vehicle>();

                //Search by Vehicle Type
                if (choice == 1)
                {
                    vehicleList.Sort();
                }
                // Search by Parked time
                else if (choice == 2)
                {
                    vehicleList.Sort(new ParkedTimeComparer());
                }
                // Display result
                Console.WriteLine("\n{0,-20} {1,-10} {2,-12} {3,-7} {4}",
                    "Registration No", "Name", "Type", "Weight", "Ticket No"
                );
                foreach (Vehicle v in vehicleList)
                {
                    Console.WriteLine(
                        "{0,-20} {1,-10} {2,-12} {3,-7:F1} {4}",
                        v.RegistrationNo,
                        v.Name,
                        v.Type,
                        v.Weight,
                        v.Ticket.TicketNo
                    );
                }
            }
            catch { }
        }
    }
}
```

```

        }
    }
    // Invalid inputs
    catch (FormatException)
    {
        Console.WriteLine("Invalid input format. Please check numeric and
date values.");
    }
    // Null or empty arguments
    catch (ArgumentException ex)
    {
        Console.WriteLine(ex.Message);
    }
    // Some other Exceptions
    catch (Exception)
    {
        Console.WriteLine("An unexpected error occurred.");
    }
}
}
}

```

Vehicle.cs

```

namespace Requirement_5
{
    // a) Create a Class Vehicle
    internal class Vehicle : IComparable<Vehicle> // d) The Vehicle class should
implement the IComparable interface
    {
        // Private fields to store Vehicle details
        private string _registrationNo;
        private string _name;
        private string _type;
        private double _weight;
        private Ticket _ticket;

        // Public property to get/set registration number
        public string RegistrationNo
        {
            get { return _registrationNo; }
            set { _registrationNo = value; }
        }

        // Public property to get/set vehicle name
        public string Name
        {
            get { return _name; }
            set { _name = value; }
        }

        // Public property to get/set vehicle type
        public string Type
        {
            get { return _type; }
            set { _type = value; }
        }

        // Public property to get/set vehicle weight
        public double Weight
    }
}

```

```

    {
        get { return _weight; }
        set { _weight = value; }
    }

    // Public property to get/set ticket details
    public Ticket Ticket
    {
        get { return _ticket; }
        set { _ticket = value; }
    }

    // Parameterized constructor to initialize all vehicle attributes
    public Vehicle(string? registrationNo, string? name, string? type, double
weight, Ticket ticket)
    {
        _registrationNo = registrationNo;
        _name = name;
        _type = type;
        _weight = weight;
        _ticket = ticket;
    }

    // b) Create Vehicle
    public static Vehicle CreateVehicle(string detail)
    {
        // Split the input string
        string[] data = detail.Split(',');

        // Vehicle details
        string registrationNo = data[0];
        string name = data[1];
        string type = data[2];
        double weight = double.Parse(data[3]);

        // Ticket details
        string ticketNo = data[4];
        DateTime parkedTime = DateTime.Parse(data[5]);
        double cost = double.Parse(data[6]);

        // Create Ticket object
        Ticket ticket = new Ticket(ticketNo, parkedTime, cost);

        // Create and return Vehicle object
        return new Vehicle(registrationNo, name, type, weight, ticket);
    }

    // IComparable implementation
    public int CompareTo(Vehicle other)
    {
        if (other == null)
            return 1;

        // Since all weights are unique, direct comparison is sufficient
        return this._weight.CompareTo(other._weight);
    }
}

```

Ticket.cs

```
namespace Requirement_5
{
    // c) Create a Class Ticket
    internal class Ticket
    {
        // Private field to store ticket details
        private string _ticketNo;
        private DateTime _parkedTime;
        private double _cost;

        // Public property to get/set ticket number
        public string TicketNo
        {
            get { return _ticketNo; }
            set { _ticketNo = value; }
        }

        // Public property to get/set parked time
        public DateTime ParkedTime
        {
            get { return _parkedTime; }
            set { _parkedTime = value; }
        }

        // Public property to get/set parking cost
        public double Cost
        {
            get { return _cost; }
            set { _cost = value; }
        }

        // Parameterized constructor to initialize ticket details
        public Ticket(string ticketNo, DateTime parkedTime, double cost)
        {
            _ticketNo = ticketNo;
            _parkedTime = parkedTime;
            _cost = cost;
        }
    }
}
```

ParkedTimeComparer.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Requirement_5
{
    // e) Create a class parkedTimeComparer which implements IComparer interface
    internal class ParkedTimeComparer : IComparer<Vehicle>
    {
        // Compare vehicles based on ParkedTime
        public int Compare(Vehicle x, Vehicle y)
        {
            if (x == null || y == null)
                return 0;
            // All parkedTime values are unique
            return x.Ticket.ParkedTime.CompareTo(y.Ticket.ParkedTime);
        }
    }
}
```



```

    }
}

```

Outputs:

Sort By Parked Time

```

Enter the number of vehicles: 5

Enter Vehicle Details:
MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75
GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,10-05-2018 05:16:26,75
WB 91 F 1649,Vento,FourWheeler,530,B4-035,10-05-2018 20:45:27,300
PN 67 FK 0065,Polo,FourWheeler,543,B4-041,11-05-2018 21:35:56,150
GJ 83 AX 0545,Ciaz,FourWheeler,530,B4-030,10-05-2018 08:25:33,150

Enter a type to sort:
1.Sort by weight
2.Sort by parked time
2

Registration No      Name      Type      Weight  Ticket No
GA 45 RF 9515      Duke      TwoWheeler  240.0    A1-024
GJ 83 AX 0545      Ciaz      FourWheeler  530.0    B4-030
WB 91 F 1649      Vento     FourWheeler  530.0    B4-035
MH 23 F 7856      Gixer     TwoWheeler  221.0    A1-013
PN 67 FK 0065      Polo      FourWheeler  543.0    B4-041

```

Sort By Vehicle Weight

```

Enter the number of vehicles: 5

Enter Vehicle Details:
WB 91 F 1649,Vento,FourWheeler,530,B4-035,10-05-2018 20:45:27,300
PN 67 FK 0065,Polo,FourWheeler,543,B4-041,11-05-2018 21:35:56,150
GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,10-05-2018 05:16:26,75
CH 51 P 9636,Ninja,TwoWheeler,287,A1-015,12-05-2018 15:05:30,100
KL 75 F 9562,Pulsar,TwoWheeler,200,A1-032,12-05-2018 14:25:23,50

Enter a type to sort:
1.Sort by weight
2.Sort by parked time
1

Registration No      Name      Type      Weight  Ticket No
KL 75 F 9562      Pulsar    TwoWheeler  200.0    A1-032
GA 45 RF 9515      Duke      TwoWheeler  240.0    A1-024
CH 51 P 9636      Ninja     TwoWheeler  287.0    A1-015
WB 91 F 1649      Vento     FourWheeler  530.0    B4-035
PN 67 FK 0065      Polo      FourWheeler  543.0    B4-041

```

ParkingLot – Requirement-6

Code:

Program.cs

```
// Parking Lot - Requirement-6

namespace Requirement_6
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                List<Vehicle> vehicleList = new List<Vehicle>();

                // Read number of vehicles
                Console.Write("Enter the number of vehicles: ");
                int.TryParse(Console.ReadLine(), out int n);

                // Read vehicle details
                Console.WriteLine("\nEnter Vehicle Details: ");
                for (int i = 0; i < n; i++)
                {
                    string input = Console.ReadLine();
                    Vehicle v = Vehicle.CreateVehicle(input);
                    vehicleList.Add(v);
                }
                // TypeWise Count Dictionary
                SortedDictionary<string, int> vehicles =
                Vehicle.TypeWiseCount(vehicleList);
                Console.WriteLine("\n{0,-15} {1}\n", "Type", "No. of Vehicles");
                foreach (var v in vehicles)
                {
                    Console.WriteLine("{0,-15} {1}", v.Key, v.Value);
                }
            }
            // Invalid inputs
            catch (FormatException)
            {
                Console.WriteLine("Invalid input format. Please check numeric and
date values.");
            }
            // Some other Exceptions
            catch (Exception)
            {
                Console.WriteLine("An unexpected error occurred.");
            }
        }
    }
}
```

Vehicle.cs

```
using System.Security.Claims;

namespace Requirement_6
{
    // a) Create a Class Vehicle
```

```

internal class Vehicle
{
    // Private fields to store Vehicle details
    private string _registrationNo;
    private string _name;
    private string _type;
    private double _weight;

    // Public property to get/set registration number
    public string RegistrationNo
    {
        get { return _registrationNo; }
        set { _registrationNo = value; }
    }

    // Public property to get/set vehicle name
    public string Name
    {
        get { return _name; }
        set { _name = value; }
    }

    // Public property to get/set vehicle type
    public string Type
    {
        get { return _type; }
        set { _type = value; }
    }

    // Public property to get/set vehicle weight
    public double Weight
    {
        get { return _weight; }
        set { _weight = value; }
    }

    // Default constructor
    public Vehicle()
    {
        _registrationNo = string.Empty;
        _name = string.Empty;
        _type = string.Empty;
        _weight = 0.0;
    }

    // Parameterized constructor
    public Vehicle(string? registrationNo, string? name, string? type, double
weight)
    {
        _registrationNo = registrationNo;
        _name = name;
        _type = type;
        _weight = weight;
    }

    // b) Create the following static method in Vehicle class,
    // Create Vehicle
    public static Vehicle CreateVehicle(string detail)
    {
        // Split the input string
        string[] data = detail.Split(',');

        // Vehicle details

```

```

        string registrationNo = data[0];
        string name = data[1];
        string type = data[2];
        double weight = double.Parse(data[3]);

        // Create and return Vehicle object
        return new Vehicle(registrationNo, name, type, weight);
    }

    // Type-wise count of vehicles
    public static SortedDictionary<string, int>TypeWiseCount(List<Vehicle>
vehicleList)
    {
        SortedDictionary<string, int> result =new SortedDictionary<string,
int>();
        foreach (Vehicle v in vehicleList)
        {
            if (result.ContainsKey(v.Type))
                result[v.Type]++;
            else
                result[v.Type] = 1;
        }
        return result;
    }
}

```

Outputs:

Output-1

```

Enter the number of vehicles: 6

Enter Vehicle Details:
MP 01 LK 0001,Hornet,TwoWheeler,163
GA 45 RF 9515 ,Auto , ThreeWheeler,240
CH 51 P 9636 ,Ninja ,TwoWheeler,287
UP 12 FV 4681,Rapid,FourWheeler,684
WB 91 F 1649,Vento,FourWheeler,530
PN 67 FK 0065,Polo,FourWheeler,543

Type                No. of Vehicles

ThreeWheeler        1
FourWheeler          3
TwoWheeler           2

```

Output-2

Enter the number of vehicles: 6

Enter Vehicle Details:

MP 01 LK 0001,Hornet,TwoWheeler,163
GA 45 RF 9515 ,Duke ,TwoWheeler,240
CH 51 P 9636 ,Ninja ,TwoWheeler,287
UP 12 FV 4681,Rapid,FourWheeler,684
WB 91 F 1649,Vento,FourWheeler,530
PN 67 FK 0065,Polo,FourWheeler,543

Type	No. of Vehicles
FourWheeler	3
TwoWheeler	3